

MODULE 5.1 · Concept 7 of 7

Planning System Implementation

Hands-on: build a plan-and-execute agent that decomposes, plans, executes, and replans — 30 min lab

30 min

Duration

B3 – Apply

Bloom's

L3 – Advanced

Level

TH-000020

Prerequisite

AI-AG-IN-CD-000002

Prof. Sashikumaar Ganesan · IISc Bangalore · AhamX Bodhi

SECTION 1 — SETUP

Building a Plan-and-Execute Agent

From goal to plan to execution to replan-on-failure

Lab Overview: What We're Building

A planning agent that decomposes, generates plans, executes, and repairs on failure

Planner LLM	ChatOpenAI(model='gpt-4o')	Generates and repairs plans from goals
Executor LLM	ChatOpenAI(model='gpt-4o-mini')	Executes individual plan steps using tools
Plan Parser	StructuredOutputParser	Converts LLM plan output into structured steps
Tools	search + calc + file_write	Real-world actions the executor can take
Replan Logic	if step_failed: replan()	Detects failures and triggers plan repair
Logger	PlanLogger class	Tracks plan, status, tokens, cost per step

Key pattern: Planner uses GPT-4o (\$0.03) for quality; Executor uses GPT-4o-mini (\$0.001) for cost. Tiered routing.

Step 1: Environment Setup

Install dependencies and configure models

```
# Terminal
pip install langchain langchain-openai tavily-python

# Python
from langchain_openai import ChatOpenAI
from langchain.tools import Tool
from langchain.output_parsers import StructuredOutputParser, ResponseSchema
import json

# Models: planner (quality) vs executor (cost)
planner_llm = ChatOpenAI(model="gpt-4o", temperature=0)
executor_llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
```

Step 2: Define the Planner + Executor Models

Separate LLMs for planning vs execution — tiered routing

Planner (GPT-4o — \$0.03/call)

Used for:

- Generating the initial plan
- Decomposing complex goals into steps
- Repairing plans when steps fail
- Verifying plan before execution

Why GPT-4o: Planning requires the best reasoning quality. Worth the extra cost because a bad plan wastes execution budget.

Executor (GPT-4o-mini — \$0.001/call)

Used for:

- Executing individual plan steps
- Calling tools with correct parameters
- Interpreting tool results
- Simple reasoning within each step

Why mini: Each step is pre-planned. The executor just follows instructions. 30× cheaper than GPT-4o per step.

Step 3: Writing the Planning Prompt

Structured output: numbered steps with dependencies and expected outputs

```
PLANNING_PROMPT = """
You are a planning agent. Given a user goal, create a step-by-step plan.

Rules:
1. Each step must be completable with ONE tool call
2. Specify dependencies between steps
3. Mark steps that can run in parallel
4. Stay within the stated budget/constraints

Output format (JSON array):
[
  {"step": 1, "description": "...", "tool": "web_search",
   "depends_on": [], "parallel_with": [2,3],
   "expected_output": "..."},
  ...
]

Available tools: web_search, calculator, code_executor, file_writer
User goal: {goal}
"""
```

Step 4: Parsing the Plan

Extract structured steps from LLM output

```
def generate_plan(goal: str) -> list:
    """Generate a structured plan from a goal."""
    response = planner_llm.invoke(PLANNING_PROMPT.format(goal=goal))

    # Parse JSON from response
    text = response.content
    # Strip markdown fences if present
    text = text.replace("```json", "").replace("```", "").strip()
    plan = json.loads(text)

    # Validate plan structure
    for step in plan:
        assert "step" in step and "tool" in step
        assert step["tool"] in AVAILABLE_TOOLS

    return plan

# Generate plan
plan = generate_plan("Research and summarize India's FY2025 budget highlights")
for step in plan:
    print(f"  Step {step['step']}: {step['description']}")
```

SECTION 2 — EXECUTE

Running the Plan Step by Step

Execute each step, handle results, detect failures

Step 5: Define Execution Tools

Search, calculator, and file writer for the executor

```
# Tool definitions (same pattern as C15)
tools = [
    Tool(name="web_search", func=web_search_with_retry,
          description="Search web for current information"),
    Tool(name="calculator", func=safe_calculator,
          description="Evaluate math expressions"),
    Tool(name="file_writer", func=write_file,
          description="Write content to a file. Args: filename, content"),
]

# Build executor as a ReAct agent (for each individual step)
from langchain.agents import create_react_agent, AgentExecutor

executor_agent = AgentExecutor(
    agent=create_react_agent(executor_llm, tools, react_prompt),
    tools=tools, max_iterations=5, verbose=True
)
```

Step 6: The Execution Loop

Iterate through plan, execute each step, collect results

```
def execute_plan(plan: list) -> dict:
    """Execute each step of the plan sequentially."""
    results = {}

    for step in plan:
        step_num = step["step"]
        print(f"\n=== Executing Step {step_num}: {step['description']} ===")

        # Check dependencies are met
        for dep in step.get("depends_on", []):
            if dep not in results or results[dep]["status"] == "failed":
                print(f" ⚠️ Dependency step {dep} not met. Triggering replan.")
                return {"status": "replan_needed", "failed_at": step_num, "results": results}

        # Execute step using ReAct executor
        try:
            result = executor_agent.invoke({"input": step["description"]})
            results[step_num] = {"status": "success", "output": result["output"]}
            print(f" ✓ Step {step_num} completed: {result['output'][:100]}...")
        except Exception as e:
            results[step_num] = {"status": "failed", "error": str(e)}
            print(f" ✖️ Step {step_num} failed: {e}")

    return {"status": "complete", "results": results}
```

Worked Example: India FY2025 Budget

Generated plan: 5 steps with execution trace

Goal: "Research and summarize India's FY2025 budget highlights"

#	Description	Tool	Status	Key Result
1	Search 'India FY2025 budget key highlights'	web_search	✓ Done	Revenue ₹33.4L Cr, Fiscal deficit 4.4%
2	Search 'India FY2025 budget tax changes'	web_search	✓ Done	New tax slabs, standard deduction ₹75K
3	Search 'India FY2025 infrastructure spending'	web_search	✓ Done	₹11.1L Cr capex, rail/road focus
4	Synthesize findings into summary	LLM reasoning	✓ Done	3-paragraph summary generated
5	Write summary to budget_summary.md	file_writer	✓ Done	File created (1.2 KB)

Total: 5 steps, 3 tool calls, ~2000 tokens, ~\$0.004. Steps 1–3 could run in parallel (1.5 sec instead of 4.5 sec).

Step 7: Execution Trace – Steps 1-3

The first three searches complete successfully

=== Executing Step 1: Search "India FY2025 budget key highlights" ===

Thought: I should search for the main budget announcements.

Action: `web_search(query="India Union Budget 2025 highlights")`

Observation: "Revenue receipts ₹33.4 lakh crore, fiscal deficit target 4.4%..."

✓ Step 1 completed.

=== Executing Step 2: Search "India FY2025 budget tax changes" ===

Thought: Need tax-specific changes from the budget.

Action: `web_search(query="India budget 2025 income tax changes")`

Observation: "New tax regime: nil tax up to ₹12L, standard deduction ₹75K..."

✓ Step 2 completed.

=== Executing Step 3: Search "India FY2025 infrastructure spending" ===

Thought: Infrastructure is a key budget theme.

Action: `web_search(query="India budget 2025 infrastructure capex")`

Observation: "Capital expenditure ₹11.1 lakh crore, railways ₹2.5L Cr..."

✓ Step 3 completed.

SECTION 3 — HARDEN

Failure Handling and Quality Evaluation

Making the planning agent production-ready

Step 8: Handling Step Failures — Replan on Error

Detect failure, preserve completed work, regenerate remaining steps

```
def replan_from_failure(original_goal, completed_results, failed_step):
    """Replan remaining steps after a failure."""
    replan_prompt = f"""
    Original goal: {original_goal}

    Completed steps and results:
    {json.dumps(completed_results, indent=2)}

    Step {failed_step} FAILED. Generate a new plan for the REMAINING
    steps only. Do not repeat completed steps. You may need to find
    an alternative approach for the failed step.

    Output JSON array of new steps (same format as before).
    """
    return generate_plan_from_prompt(replan_prompt)

# Usage in execution loop:
if step_result["status"] == "failed":
    new_plan = replan_from_failure(goal, results, step_num)
    execute_plan(new_plan) # Continue with repaired plan
```

Step 9: Plan Logging

Structured logs for monitoring, debugging, and cost tracking

```
class PlanLogger:
    def __init__(self):
        self.logs = []

    def log_step(self, step_num, description, tool, status, result, tokens, cost):
        entry = {
            "step": step_num,
            "description": description,
            "tool": tool,
            "status": status,
            "result": result[:200], # Truncate for logging
            "tokens": tokens,
            "cost": cost,
            "timestamp": datetime.now().isoformat()
        }
        self.logs.append(entry)

    def summary(self):
        total_tokens = sum(l["tokens"] for l in self.logs)
        total_cost = sum(l["cost"] for l in self.logs)
        completed = sum(1 for l in self.logs if l["status"] == "success")
        return f"Steps: {completed}/{len(self.logs)} | Tokens: {total_tokens} | Cost: ${total_cost:.4f}"
```

Step 10: Evaluating Plan Quality

Did it work? Was it efficient? What did it cost?

Metric	How to Measure	Good Target	Our Budget Example
Completion rate	Steps completed / total steps	$\geq 90\%$	5/5 = 100%
Result relevance	LLM-as-judge scores each result	$\geq 4/5$ average	4.5/5
Total tokens	Sum across all steps	< model context limit	~2000 tokens
Total cost	Sum of all LLM + tool costs	< \$0.01 for simple tasks	\$0.004
Wall-clock time	End-to-end execution time	< 30 sec for 5-step plans	~8 sec
Replan count	Number of replans triggered	0–1 for well-designed plans	0 (no failures)

Key Takeaways

The essential points from this concept

Planner (GPT-4o) + Executor (mini): tiered routing saves cost while maintaining plan quality.

Structured planning prompts with JSON output + atomicity rules produce reliable plans.

Execution loop: iterate steps, check dependencies, execute with ReAct, handle failures.

Plan repair preserves completed work — only replans remaining steps from current state.

This pattern is the foundation for all production planning agents in Module 9 (LLMOps).

Next: Sub-Module 5.4 — Agent Communication, MCP, and Context

AI-AG-IN-CD-000002 Complete

Planning System Implementation

Concept 7 of 8

Advanced Certification in Agentic & Generative AI · IISc Bangalore

Prof. Sashikumaar Ganesan · AhamX Bodhi